SYNOPSIS

ON

PhishEye: AI & ML based Cyberguard

Submitted in

Partial Fulfillment of the requirements for the Award of the Degree

of

Bachelor of Technology

In

Computer Science and Engineering

by

(Project Id: 26_CS_2L_19)

Hritik Kumar (2501640100427)
Kirt Raj Dixit (2501640100468)

Under the supervision of

Prasun Tripathi

(Assistant Professor)

Pranveer Singh Institute of Technology
Kanpur - Agra - Delhi National Highway - 19, Bhauti - Kanpur - 209305

1. Introduction

PhishEye: AI & ML based Cyberguard is a hybrid, AI-assisted phishing detection system that helps a person decide whether a link or a message is safe before acting on it. Phishing is the most common entry point for online fraud: a message on SMS, WhatsApp or e-mail imitates a bank, a courier or a government office and pushes the reader to open a link and type a password, an OTP, a UPI PIN or card details. The fake page often shows a valid padlock, a familiar logo and an address that differs from the real one by a single letter, so a non-technical user cannot tell it apart. Browser blocklists react only after a site is reported and say nothing about the wording that lured the user there.

PhishEye addresses this gap with a software-only system made of a web console, a REST service, a local database and a Chrome extension. The user submits a web address, pastes a suspicious message, or uploads a QR-code image. Three analysis engines run on the same input: URL and domain heuristics (lookalike brand names, newly registered domains, suspicious endings, missing HTTPS), live lookups in public threat-intelligence feeds, and natural-language rules that detect psychological pressure such as urgency, fear, authority and OTP or password requests, in English and Hinglish. A weighted fusion step combines the signals into a 0-100 risk score and one of four levels (LOW, MEDIUM, HIGH, CRITICAL). The result is explained in a Simple View for ordinary users and a Technical View for operators, and the extension applies the same decision to block a dangerous page before it opens. The current prototype is rule- and feed-driven; training a supervised machine-learning classifier on labelled phishing data is planned as future scope (Section 4.8).

Key terms: phishing - impersonation of a trusted party to steal credentials or money; lookalike domain - an address crafted to resemble a genuine brand; threat intelligence - publicly shared lists of confirmed malicious addresses; psychological manipulation cues - wording that pressures the reader to act without checking; explainability - presenting the reasons behind a verdict in plain language; risk level - the category assigned to a score so that a non-expert can act on it.

2. Project Objective

The project sets out to achieve the following outcomes:

1. To detect phishing and lookalike-domain web addresses, whether typed directly, pasted inside a message, or encoded in a QR image, and to express the result as a 0-100 risk score with a named level.

2. To identify social-engineering pressure in message text - urgency, fear, authority, prize bait and requests for OTPs or passwords - in both English and Hinglish.
3. To cross-check every address against public threat-intelligence feeds so that a confirmed phishing report always produces a high score, whatever the other signals say.
4. To explain each verdict in two forms: a Simple View written for ordinary users and a Technical View listing the signals, tags and feed matches for operators.
5. To protect users at the moment of risk through a Chrome extension that scans a link before the page opens and keeps high-risk or unverifiable destinations closed.
6. To maintain a secure, per-user history of scans, community reports and feedback in a lightweight local database, with hashed passwords and token-based login.
7. To design the scoring pipeline so that a supervised machine-learning classifier can be added later as one more signal without changing the user-facing workflow.

The primary beneficiary is a student, a first-time digital-banking user, or anyone who receives OTP and KYC lures and does not read certificate details: they get a clear answer and a short reason before they click. Console operators gain a searchable history, statistics and community reports that reveal recurring scam domains. The system never reads private inboxes or phone SMS; it analyses only what the user submits or the page the user is about to visit.

Table 1: Key features of PhishEye

Feature	What it does for the user
Multi-input scanning	Accepts a URL, pasted SMS/WhatsApp/e-mail text or a QR image; scores every link found
URL and domain heuristics	Flags lookalike brands, new domains, suspicious endings, shorteners, missing HTTPS
Threat-intelligence lookup	Checks OpenPhish, URLhaus, PhishTank, Phishing Army (Safe Browsing with a key)
Psychological-cue detection	Finds urgency, fear, authority, greed bait, OTP/password requests (English, Hinglish)
Weighted risk fusion	Combines all signals into a 0-100 score and a LOW / MEDIUM / HIGH / CRITICAL level
Simple and Technical View	Plain-language warning for everyone; detailed evidence for operators
Chrome extension gate	Scans a link before the page opens; blocks HIGH, CRITICAL and unverifiable pages
History and community reports	Stores scans per user; users can mark a domain scam, risky or safe as one extra signal

3. Feasibility Study

3.1 Technical Feasibility

The system is built from mature open-source components - Python 3.12 with FastAPI, SQLite, React 19 with Vite, OpenCV and the Chrome Manifest V3 platform - all already integrated and running. No graphics card or special server is needed because detection uses rules, lookups and arithmetic rather than a neural network. Threat feeds are reached over HTTPS with time-outs and a short local cache.

3.2 Operational Feasibility

A user registers, opens the Scan page, submits a link, message or image, and reads the Simple View first. The extension installs through Chrome's developer mode, so no store listing is needed. If the service cannot be reached, the extension keeps the page closed: an unknown page is not a safe page.

3.3 Economic Feasibility

Every library is free and open source and the database is a local file with no licence fee. The threat feeds are public; Google Safe Browsing is optional and off without a key. A free hosting tier suffices for demonstration and development runs on a student laptop, so there is no mandatory recurring cost.

3.4 Schedule Feasibility

The work is organised into six phases over a twelve-week semester plan (Table 5). The engines, scoring, console, extension and tests already exist as a working prototype, so the remaining effort concentrates on evaluation, documentation and future-scope items, with slack left for review feedback.

3.5 Legal and Ethical Feasibility

The system stores only an account (user name, e-mail, hashed password) and the addresses or results the user submits. It never captures passwords, OTPs or card numbers from web pages; the extension does not read form fields, cookies or phone messages; QR images are decoded in memory and not saved. The scanner is blocked from visiting internal network addresses, and community reports are shown as user opinions.

Table 2: Feasibility summary

Type	Assessment
Technical	Feasible - proven open-source stack, already integrated; no GPU needed

Type	Assessment
Operational	Feasible - simple web workflow; fails closed when the service is unreachable
Economic	Feasible - no licence or mandatory hosting cost; paid feed optional
Schedule	Feasible - prototype complete; twelve-week plan with slack for write-up
Legal / ethical	Feasible - minimal personal data, hashed passwords, no inbox access

4. Methodology / Planning of Work

The project follows an incremental, phase-wise methodology; each phase produces a reviewable artefact, and the figures and tables below follow the phase they describe.

4.1 Phase 1 - Problem Study and Requirement Analysis

Real phishing lures (bank KYC, courier fee, electricity disconnection, prize and job scams) were collected from public awareness material and studied for technical tricks in the address and psychological tricks in the wording. The study produced the requirements in Section 2, the four-level risk scale and the rule that an unverifiable result must never be reported as safe.

4.2 Phase 2 - System Design

The architecture is layered (Fig. 1). Clients talk to a single REST service, which runs the analysis engines, hands their signals to the risk-decision engine, builds the two explanation views and stores results in SQLite. The QR decoder feeds the decoded address into the same URL analysis path as a typed link.

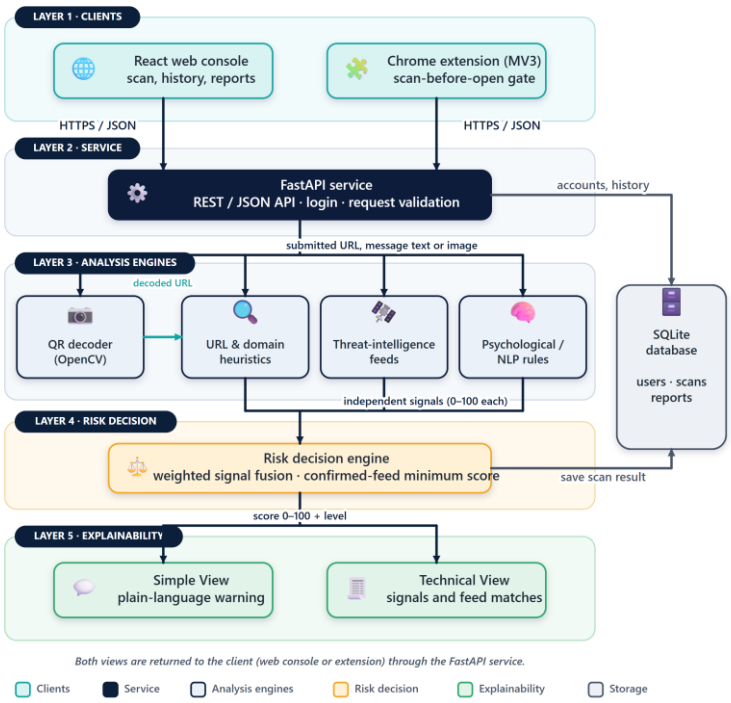


Fig. 1: Layered system architecture of PhishEye.

Fig. 2 shows the system as one process exchanging data with the user, the extension and the feed services; Fig. 3 expands it into six numbered processes and two data stores, D1 (SQLite database) and D2 (local feed cache).

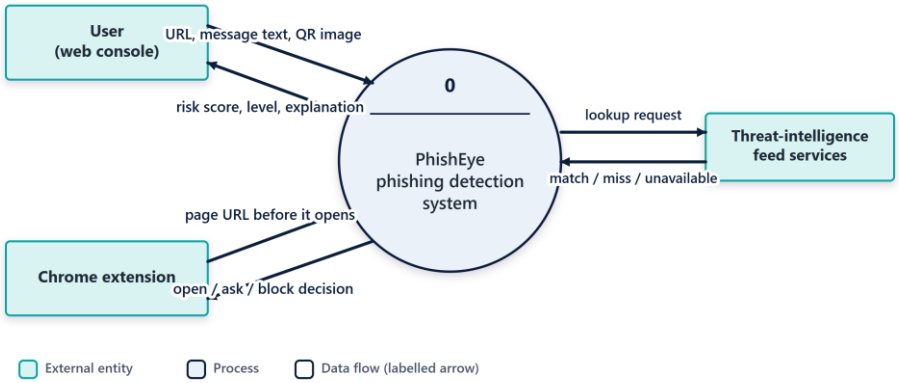


Fig. 2: Data-flow diagram, Level 0 (context diagram).

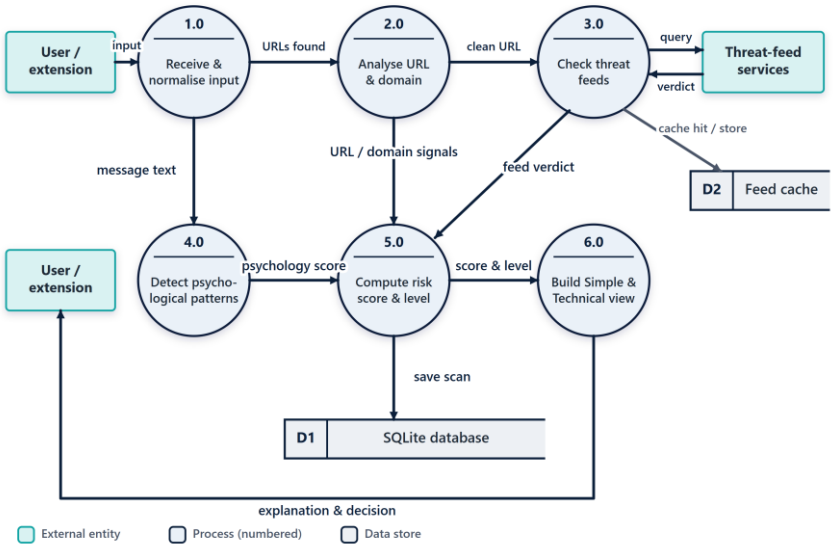


Fig. 3: Data-flow diagram, Level 1 with numbered processes and data stores.

The use-case view (Fig. 4) identifies three actors. A visitor can scan a link, message or QR image without logging in; viewing history, reporting a domain and giving feedback require a signed-in account. The extension acts as an automated client; the feed services are external systems consulted during a scan.

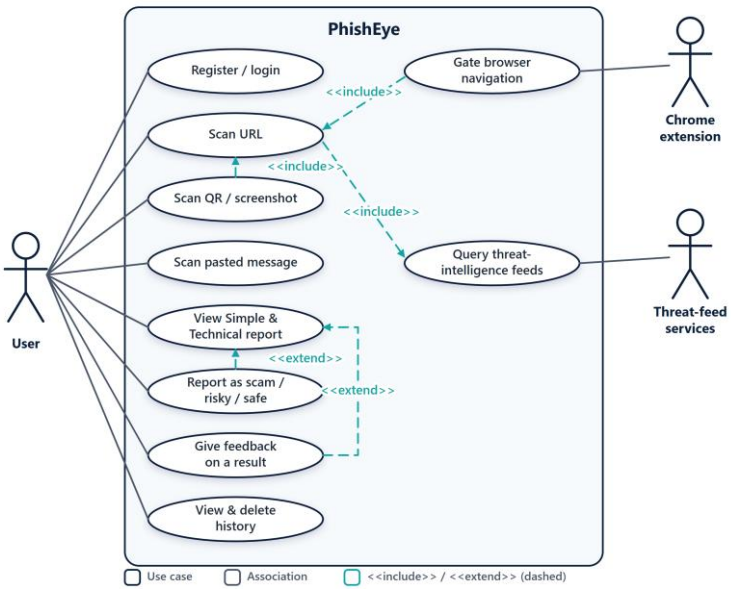


Fig. 4: Use-case diagram of PhishEye.

The database design (Fig. 5) keeps one table per concern. Solid lines mark relationships enforced by a declared foreign key; dashed lines mark references stored as plain identifier columns and maintained by the application. The domain-reputation table is independent and keyed by domain name.

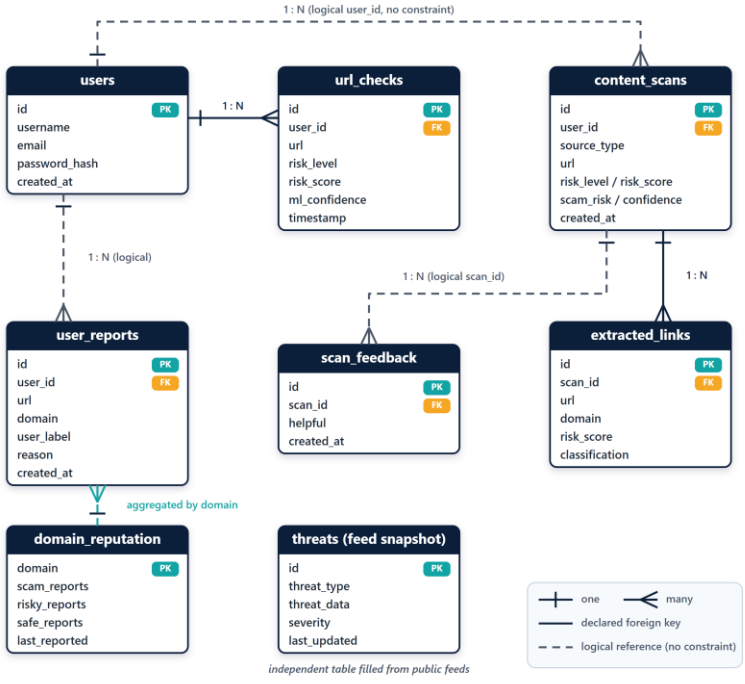


Fig. 5: Entity-relationship diagram of the SQLite database (crow's-foot notation).

4.3 Phase 3 - Implementation of the Analysis Engines

Three engines were implemented and tested independently. The URL and domain engine normalises the address, safely expands shortened links, checks whether the domain exists and how recently it was registered (under 30 days adds strongly to the score), inspects the

certificate, and matches the host against brand and lookalike patterns. The threat-intelligence engine queries each feed with a time-out and records match, no match or unavailable, so a feed outage is visible rather than treated as clean. The psychology engine applies phrase rules in ten categories (urgency, fear, authority, coercion, financial pressure, credential pressure, countdown, greed bait, secrecy and emotional manipulation), each with a fixed point value; wording alone can raise a score to at most 35, so a harmless message with urgent words is never blocked outright.

4.4 Phase 4 - Risk Decision and Explanation

For content with several links, the engine signals are combined with the fixed weights in Table 3 (they sum to 1.0). For a single address, the heuristic score is compared with a minimum score tied to the strength of any feed match (exact address 85, normalised address 80, host name 72, registered domain 55) and the higher value wins, so heuristics can never cancel a confirmed report. The final score is mapped to a level (Table 4); the same decision drives both views and the extension (Fig. 6).

Table 3: Fusion weights used to combine engine signals

Signal	Weight	What it measures
URL structure	0.22	Lexical tricks in the address: lookalike words, odd characters, long paths
Domain reputation	0.16	Domain existence, age, suspicious ending, brand lookalike
Lexical score	0.10	Compact numeric summary of the URL features
Message language	0.10	Scam vocabulary and link-bait phrasing in the text
Psychological cues	0.10	Urgency, fear, authority, OTP/password pressure
Threat intelligence	0.10	Match status across the public feeds
Brand impersonation	0.08	Named bank or brand referenced by a non-official domain
Sender signals	0.05	Sender identity or header hints when supplied
Community reports	0.05	Scam / risky / safe marks from other users
Redirect behaviour	0.04	Number of redirect hops and shortener use

Table 4: Risk levels and the action taken

Level	Score range	Meaning for the user	Extension action
LOW	0 - 29	No malicious indicators found	Page opens
MEDIUM	30 - 49	Some suspicious signs; check before trusting	User is asked to confirm
HIGH	50 - 69	Strong phishing indicators	Page is blocked
CRITICAL	70 - 100	Confirmed or near-certain phishing	Page is blocked

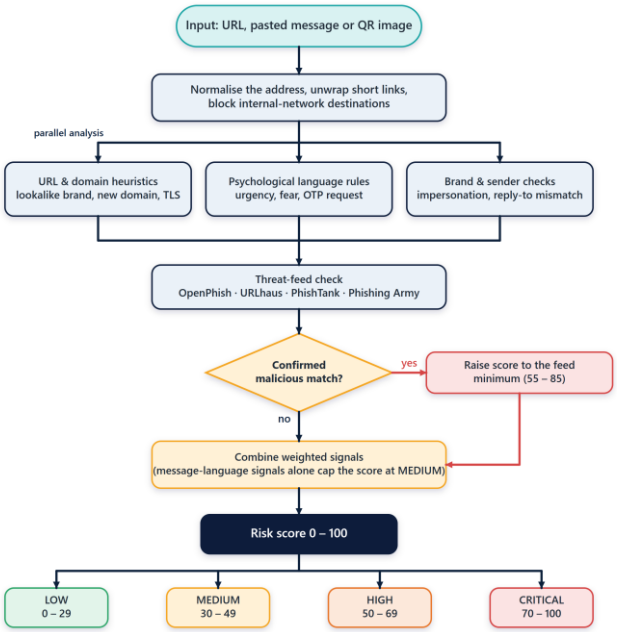


Fig. 6: Risk decision flow from input to colour-coded level.

4.5 Phase 5 - Browser Extension

The extension intercepts navigation to a new address, holds the tab, and asks the service for a verdict (Fig. 7). LOW opens the page; MEDIUM shows the warning and lets the user decide; HIGH and CRITICAL stay blocked. If the service times out or is unreachable the page also stays blocked ("unknown is not safe"). Verdicts are cached for 90 seconds to avoid repeated scans of the same address.

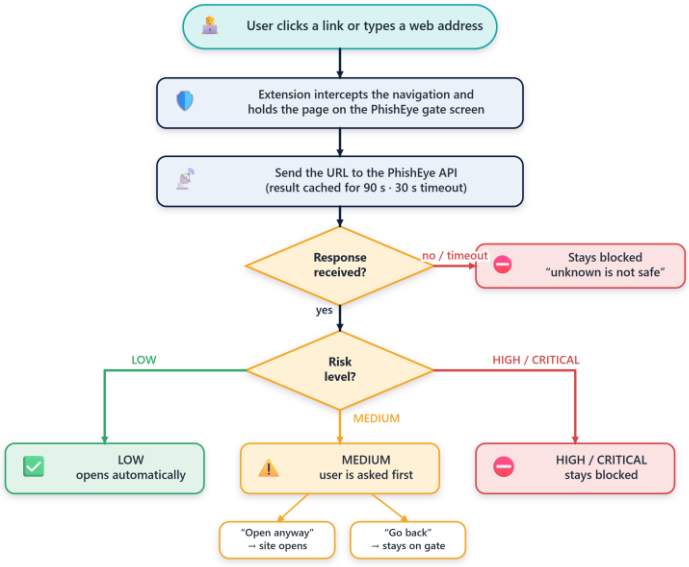


Fig. 7: Chrome extension scan-before-open gating flow.

4.6 Phase 6 - Testing, Deployment and Documentation

Automated tests cover the API routes, psychology rules, feed-status handling, QR decoding, certificate helpers and address normalisation, and a set of known phishing and legitimate

samples is re-run after every change. The service and console are deployed on a free hosting tier for demonstration. Fig. 8 shows a real scan of a lookalike banking address; Fig. 9 shows the per-user history page.

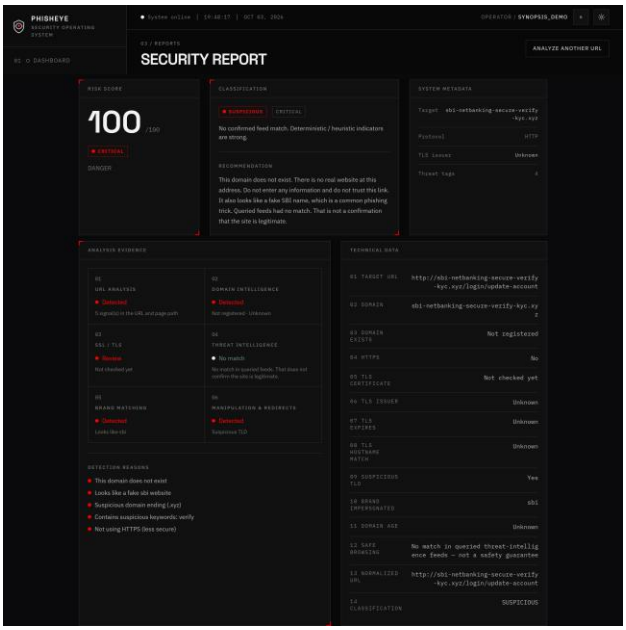


Fig. 8: Scan result page: risk score, classification, recommendation and technical evidence.

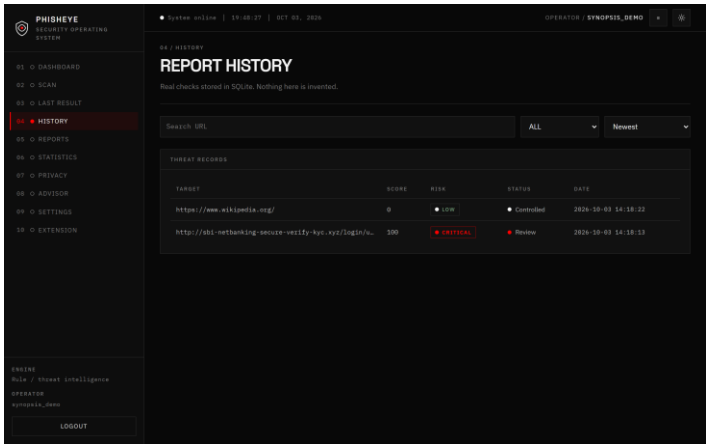


Fig. 9: Report history page listing stored scans with score, level and status.

4.7 Work Schedule

Table 5: Phase-wise work plan

Phase	Activities	Duration / Week
1. Problem study	Collect sample lures, study attack patterns, fix requirements and risk scale	Weeks 1-2
2. System design	Architecture, data-flow, use-case and database diagrams; API contract	Weeks 3-4
3. Analysis engines	URL/domain heuristics, threat-feed client with cache, psychology rules	Weeks 5-7
4. Decision and explanation	Weighted fusion, feed minimum scores, Simple and Technical views	Week 8

Phase	Activities	Duration / Week
5. Browser extension	Navigation gate, cache, fail-closed handling, install guide	Weeks 9-10
6. Testing and documentation	Automated tests, sample evaluation, deployment, synopsis and report	Weeks 11-12

4.8 Future Scope

The present prototype relies on hand-written rules and public feeds. The next step is to assemble a labelled dataset of phishing and legitimate URLs and messages (for example from PhishTank, OpenPhish and public benchmark collections), train a supervised classifier on the features the heuristics already extract, and add its probability as one more weighted signal with precision, recall and F1 reported on a held-out set. Further extensions include text recognition for screenshots, a Firefox version of the extension, and migration to a server database if multi-user load grows.

5. Tools / Technology Used

5.1 Minimum Hardware Requirements

The following configuration is sufficient to run the service, the development server, Chrome and the QR decoder together on a student laptop. No graphics card is used.

Table 6: Minimum hardware requirements

Component	Minimum requirement
Processor	64-bit dual-core, 2.0 GHz or faster (Intel Core i3 class or equivalent)
Memory	8 GB RAM (16 GB recommended when the browser, build tools and service run together)
Storage	10 GB free disk space for the code, Python environment, Node modules and the database
Graphics	Not required; integrated graphics is sufficient
Network	Internet connection for threat-feed lookups and deployment
Peripherals	Keyboard, display; a camera is not needed because QR images are uploaded as files

5.2 Minimum Software Requirements

Versions are those declared in the project's dependency files; ">=" denotes the minimum version accepted by the project and "^" the compatible range used by the front-end build.

Table 7: Software requirements and technology stack

Category	Tool / Library	Version	Purpose
Language	Python	3.12.8	Service and analysis engines

Category	Tool / Library	Version	Purpose
Web framework / server	FastAPI / Uvicorn [standard]	>=0.115.0 / >=0.30.0	REST JSON API and its server
Validation	Pydantic / pydantic-settings	>=2.7.0 / >=2.2.0	Request models and settings
Authentication	passlib[bcrypt] / bcrypt / python-jose	>=1.7.4 / 4.0.1 / >=3.3.0	Password hashing, login tokens
HTTP clients	requests / httpx	>=2.31.0 / >=0.27.0	Threat-feed and domain lookups
Image processing	opencv-python-headless / Pillow / NumPy	>=4.8.0 / >=10.0.0 / >=1.26.0	QR-code decoding from uploads
Database	SQLite (Python sqlite3 module)	bundled with Python 3.12	Accounts, scans, reports
Other back-end	python-dotenv / python-multipart / SQLAlchemy / pytest	>=1.0.0 / >=0.0.9 / >=2.0.30 / >=8.0.0	Configuration, uploads, ORM reserved for future work, tests
Front-end runtime	Node.js	22.12.0	Build tooling (deployment pin)
UI and routing	React, React DOM / react-router-dom / axios	^19.2.8 / ^7.18.2 / ^1.19.0	Web console, navigation, API calls
Charts and build	recharts, lucide-react / Vite, Tailwind CSS	^3.10.1, ^1.31.0 / ^8.2.0, ^4.3.3	Statistics charts, icons, bundling, styling
Browser extension	Chrome Extension Manifest V3	extension 1.8.0	Scan-before-open gate
Platform	Windows 10/11 or Linux; Chrome or other Chromium browser	current	Development, deployment, running the console
Editor / optional	Visual Studio Code or Cursor; Google Safe Browsing key; Tesseract OCR	-	Development; extra feed; text from screenshots

6. References

[1] R. M. Mohammad, F. Thabtah, and L. McCluskey, "Intelligent rule-based phishing websites classification," IET Information Security, vol. 8, no. 3, pp. 153-160, May 2014.

[2] R. M. Mohammad, L. McCluskey, and F. Thabtah, "Phishing Websites," UCI Machine Learning Repository, 2015. [Online]. Available: https://archive.ics.uci.edu/dataset/327/phishing+websites. Accessed: Oct. 3, 2026.

[3] J. Ma, L. K. Saul, S. Savage, and G. M. Voelker, "Beyond blacklists: Learning to detect malicious web sites from suspicious URLs," in Proc. 15th ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining (KDD), Paris, France, 2009, pp. 1245-1254.

[4] Y. Zhang, J. I. Hong, and L. F. Cranor, "CANTINA: A content-based approach to detecting phishing web sites," in Proc. 16th Int. Conf. World Wide Web (WWW), Banff, Canada, 2007, pp. 639-648.

[5] R. Dhamija, J. D. Tygar, and M. Hearst, "Why phishing works," in Proc. SIGCHI Conf. Human Factors in Computing Systems (CHI), Montreal, Canada, 2006, pp. 581-590.

[6] S. Sheng, B. Wardman, G. Warner, L. F. Cranor, J. Hong, and C. Zhang, "An empirical analysis of phishing blacklists," in Proc. 6th Conf. Email and Anti-Spam (CEAS), Mountain View, CA, USA, 2009.

[7] S. Garera, N. Provos, M. Chew, and A. D. Rubin, "A framework for detection and measurement of phishing attacks," in Proc. ACM Workshop on Recurring Malcode (WORM), Alexandria, VA, USA, 2007, pp. 1-8.

[8] A. K. Jain and B. B. Gupta, "A survey of phishing attack techniques, defence mechanisms and open research challenges," Enterprise Information Systems, vol. 16, no. 4, pp. 527-565, 2022.

[9] Anti-Phishing Working Group, "Phishing Activity Trends Reports." [Online]. Available: https://apwg.org/trendsreports/. Accessed: Oct. 3, 2026.

[10] Cisco Talos, "PhishTank: Join the fight against phishing." [Online]. Available: https://phishtank.org/. Accessed: Oct. 3, 2026.

[11] abuse.ch, "URLhaus: Malware URL exchange." [Online]. Available: https://urlhaus.abuse.ch/. Accessed: Oct. 3, 2026.

[12] OpenPhish, "OpenPhish: Phishing intelligence." [Online]. Available: https://openphish.com/. Accessed: Oct. 3, 2026.

[13] S. Ramirez, "FastAPI." [Online]. Available: https://fastapi.tiangolo.com/. Accessed: Oct. 3, 2026.

[14] Google, "Chrome Extensions: Manifest V3." [Online]. Available: https://developer.chrome.com/docs/extensions/. Accessed: Oct. 3, 2026.

[15] SQLite Consortium, "SQLite Documentation." [Online]. Available: https://www.sqlite.org/docs.html. Accessed: Oct. 3, 2026.